Universidad San Carlos de Guatemala

Organización de Lenguajes y Compiladores 2

Ing. Mario Tobar



MANUAL TÉCNICO

Mario Raúl Yancor Ocaña

201930761

Quetzaltenango, 27 de octubre de 2026

ÍNDICE

Contenido del documento **3**
Presentación, alcance y objetivos **3**
Tecnologías y requisitos **4**
Organización del repositorio **4**
Arquitectura del compilador **5**
Secuencia de compilación 5
Diagrama de clases de alto nivel 5
Léxico y gramáticas de los tres lenguajes **6**
Lenguaje Y? 6
Lenguaje Zetariano 6
Lenguaje PigLatin 6
AST y tabla de símbolos **7**
Análisis semántico y compatibilidad de tipos **7**
Equivalencia básica de tipos 7
Resultado de expresiones habituales 8
Compatibilidad en asignaciones 8
Herencia, polimorfismo y objetos **8**
Cuartetas, Stack y Heap **9**
Generación de código C y RISC-V **9**
Backend C y C3D 9
Backend RISC-V 10
Interfaz gráfica y gestión de proyectos **10**
Diagnósticos, pruebas y evidencia **10**
Regresión automática 11
Instalación, ejecución y mantenimiento **11**
Puntos de mantenimiento 11
Trazabilidad con el enunciado **12**
Conclusiones técnicas y fuentes **12**

Presentación, alcance y objetivos

Contacto 3xtrat3rr3str3D es una aplicación de escritorio desarrollada en Java para analizar proyectos que combinan los lenguajes Y?, Zetariano y PigLatin. La herramienta realiza análisis léxico, sintáctico y semántico, construye representaciones abstractas de los archivos, genera cuartetas, planifica Stack/Heap y produce código intermedio de tres direcciones y código final en C y en ensamblador RISC-V.

La evolución con respecto al Proyecto 1 es importante: el sistema dejó de ser un traductor de un archivo Latinus hacia PigLatin y ahora coordina varios archivos y traduce programas de alto nivel a código ejecutable, tomando el código intermedio como base de generación.

Validar los tres lenguajes usando gramáticas ANTLR4 y reportar diagnósticos por archivo, línea y columna.
Resolver las referencias entre archivos y verificar atributos, métodos, tipos, herencia y uso de objetos.
Emitir cuartetas y representaciones con Stack y Heap como paso previo a los generadores C y RISC-V.
Conservar una interfaz de administración de archivos, editor con coloreado y consulta de resultados.

Tecnologías y requisitos

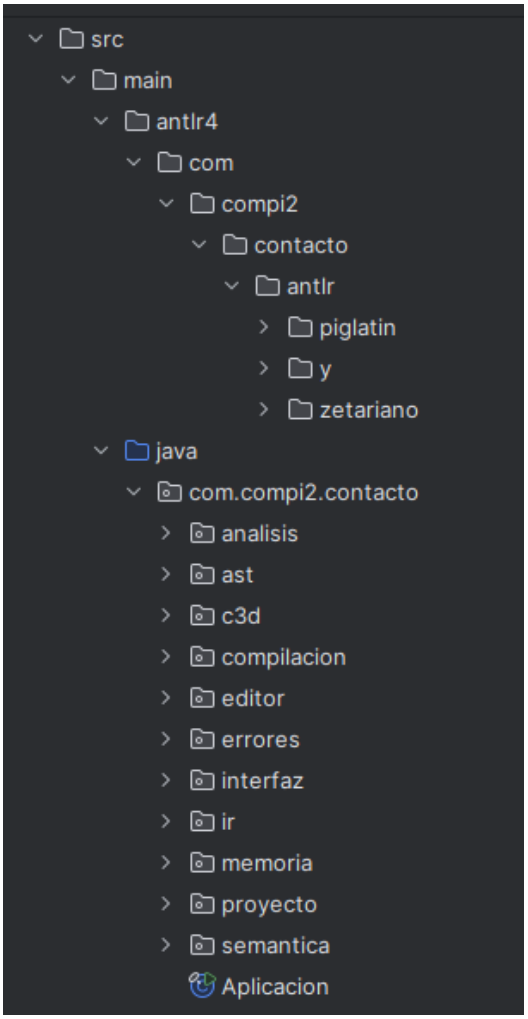
Herramienta	Versión o referencia	Función
Java SE	JDK 21	Implementación, Swing, registros (records), colecciones y archivos
Apache Maven	Configuración pom.xml	Compilación, pruebas y ejecución
ANTLR4	4.13.2	Generación de lexer, parser, listener y visitor por lenguaje
JUnit Jupiter	5.11.4	Pruebas automáticas
Java Swing	Incluido en Java	Árbol de proyecto, editor, pestañas y controles
GCC	Instalación externa	Compilar el backend C generado, cuando se requiera
RARS	Simulador RISC-V	Ejecutar archivos .s del backend RISC-V
Git/GitHub	Control de versiones	Repositorio y entrega de código y manuales

La aplicación puede iniciarse desde un entorno que disponga de Java 21 y Maven. Para ejecutar archivos .s se utiliza RARS, por ejemplo el JAR rars1_6.jar; dicho simulador no está embebido dentro de la aplicación.

```
mvn clean test
 mvn exec:java
```

El pom.xml fija maven.compiler.release=21, ANTLR 4.13.2 y JUnit 5.11.4. El plugin antlr4-maven-plugin genera tanto visitors como listeners.

Organización del repositorio



Las extensiones de archivos reconocidas son .y para Y?, .z para Zetariano y .pig para PigLatin. GestorProyecto recorre la carpeta de trabajo, identifica las extensiones y carga el contenido en UTF-8. CompiladorProyecto exige exactamente un archivo principal .pig por proyecto.

Paquete	Clases principales	Responsabilidad
proyecto	GestorProyecto, ProyectoCompilacion, ArchivoFuente	Lectura y organización del proyecto
analisis	AnalizadorArchivo, YAstBuilder, ZAstBuilder, PAstBuilder	ANTLR y construcción de AST
semantica	AnalizadorSemanticoY/Z/Pig, TablaSimbolos	Validación por lenguaje, imports y referencias
ir	Cuarteta, ProgramaIntermedio, GeneradorIntermedioProyecto	Representación intermedia
memoria	PlanificadorMemoriaProyecto, BajadorMemoriaProyecto y bajadores	Layouts, marcos y referencias en memoria
c3d	GeneradorCodigoTresDirecciones, GeneradorCodigoC, GeneradorRiscV	Backend C3D y código final
interfaz/editor	VentanaPrincipal, PanelEditor, ColoreadorAntlr	Interacción y coloreado

Arquitectura del compilador

La clase CompiladorProyecto coordina el análisis de todos los archivos, reúne diagnósticos y, únicamente cuando el proyecto es válido, genera las cuartetas y la representación de memoria. Los generadores de salida consumen esa representación sin necesidad de volver a interpretar el texto original.

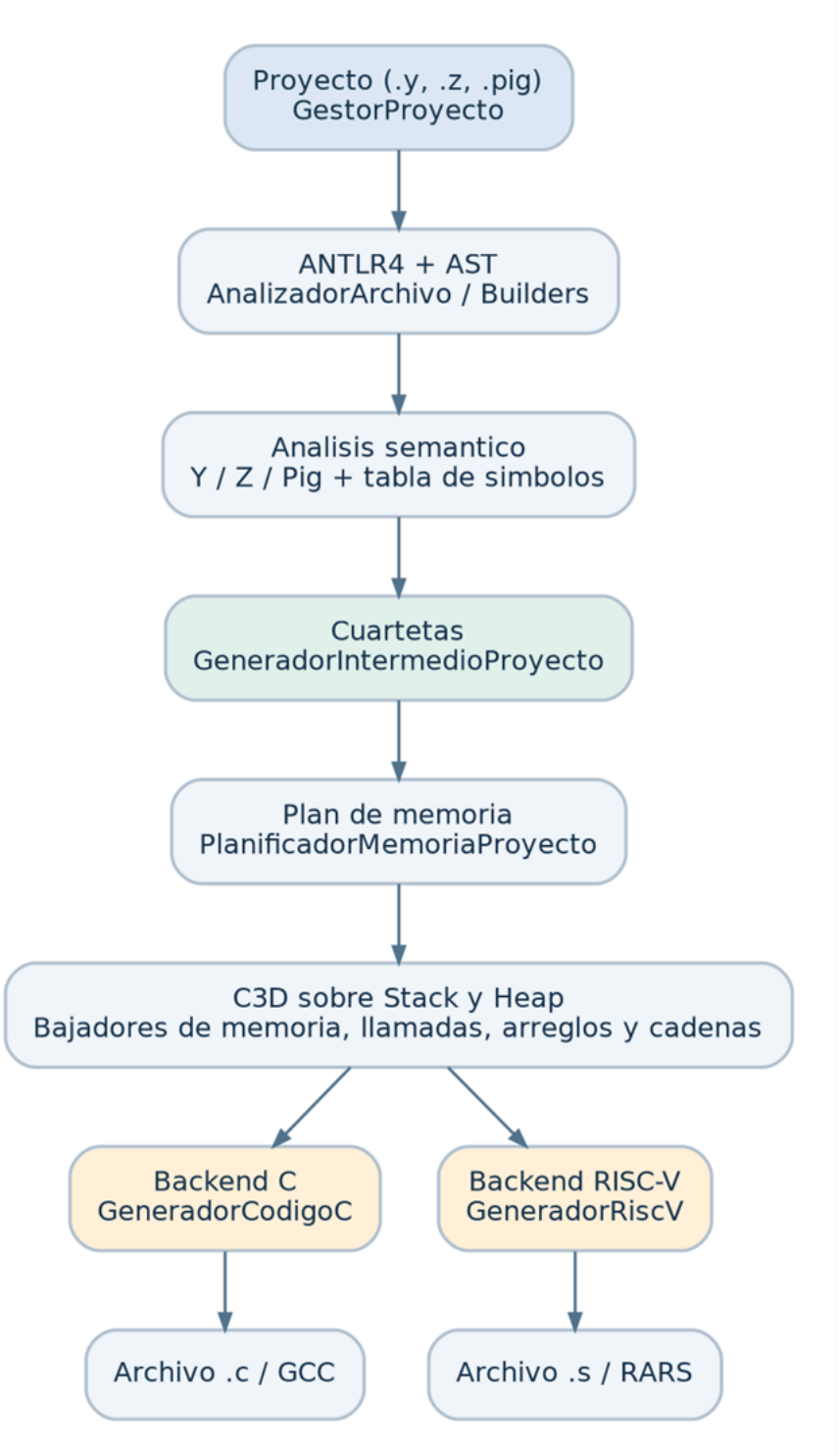


Figura 1. Flujo general desde archivos fuente hasta los backends C y RISC-V.

Secuencia de compilación

1. GestorProyecto identifica y lee los archivos .y, .z y .pig del directorio.
2. AnalizadorArchivo ejecuta el lexer y parser correspondientes y los builders producen AST por archivo.
3. AnalizadorSemanticoY, AnalizadorSemanticoZ y AnalizadorSemanticoPig comprueban símbolos, tipos, herencia e imports.
4. ConstructorTablaSimbolosProyecto reúne la información de símbolos.
5. GeneradorIntermedioProyecto transforma las sentencias en cuartetas.
6. PlanificadorMemoriaProyecto calcula marcos de Stack y distribución de objetos y estructuras en Heap.
7. BajadorMemoriaProyecto y los bajadores especializados convierten llamadas, arreglos, inicializadores y cadenas en operaciones concretas.
8. Los generadores producen código C3D, C o RISC-V; la interfaz permite consultarlo y guardar el archivo generado.

Diagrama de clases de alto nivel

La figura muestra dependencias entre clases reales y sus operaciones principales; es un diagrama resumido de los componentes centrales, no de todas las clases presentes en el repositorio.

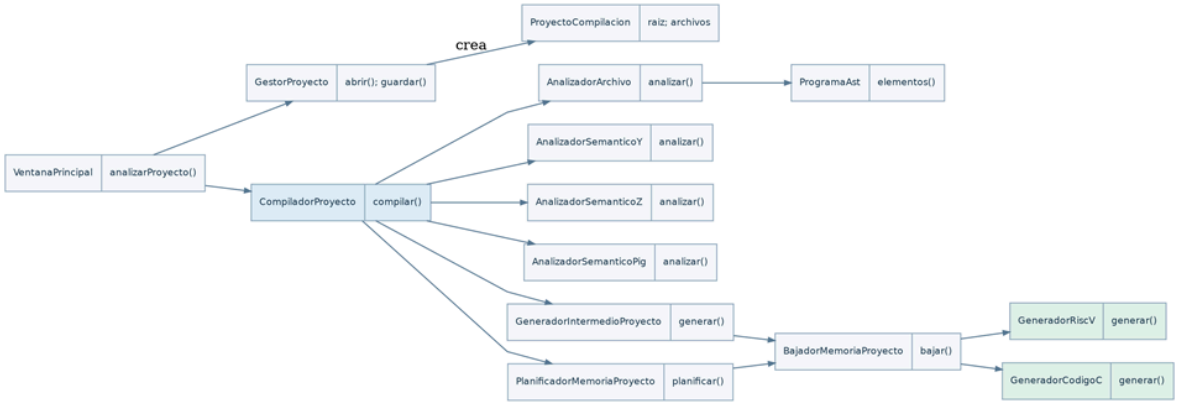


Figura 2. Diagrama de clases y dependencias principales del compilador.

Léxico y gramáticas de los tres lenguajes

Cada lenguaje posee archivos lexer y parser separados bajo src/main/antlr4. ANTLR4 reconoce tokens y estructuras gramaticales; luego el análisis semántico decide si los elementos reconocidos son válidos en el contexto del programa.

Lenguaje Y?

Y? utiliza indentación significativa. TokenizadorIndentacionY genera INDENT y DEDENT, mientras YLexer reconoce las secciones %estructuras y %funciones, tipos y palabras reservadas.

archivo : NUEVA_LINEA* seccionEstructuras? seccionFunciones NUEVA_LINEA* EOF;

```
definicionEstructura
  : KW_ESTRUCTURA IDENTIFICADOR DOS_PUNTOS finLinea
    INDENT atributoEstructura+ DEDENT;
```

La definición de estructuras es con nombre y atributos indentados; esta es la sintaxis utilizada por el proyecto. No se emplea la variante de llaves tipo C para declarar estructuras de Y?.

Categoría	Ejemplos de tokens	Función
Secciones	%estructuras, %funciones	Organizar estructuras y funciones
Control	si, entonces, sino, elegir, caso, mientras, para	Flujo y ciclos
Datos	entero, flotante, cadena, caracter, bool	Tipos básicos
Lógicos	verdadero, falso, &&, ||, !	Valores y operadores booleanos
Funciones	definir, retornar, imprimir, leer	Declarar y utilizar funciones

Lenguaje Zetariano

```
archivo : definicionClase EOF;
definicionClase
  : KW_PUBLIC? KW_CLASS IDENTIFICADOR herencia?
      LLAVE_ABRE miembroClase* LLAVE_CIERRA;
herencia : KW_EXTENDS IDENTIFICADOR;
```

Zetariano define clases, atributos, constructores y métodos. El lexer incluye public, private, protected, extends, this, super, @Override y los tipos int, double, char, boolean y String. Los métodos pueden sobreescribirse; los constructores pueden invocar super(...).

Lenguaje PigLatin

```
archivo : importacion* seccionVariables? seccionPrincipal EOF;
seccionPrincipal : SECCION_PRINCIPAL sentencia* FIN_PROGRAMA;
definicionVariable
  : tipoDato expresion? | BOOL_VERUM | BOOL_FALSUS
  | KW_NOVUS IDENTIFICADOR argumentos (DOS_PUNTOS IDENTIFICADOR)?
  | KW_NULL DOS_PUNTOS IDENTIFICADOR
  | IDENTIFICADOR inicializadorCompuesto?;
```

PigLatin es el archivo principal del proyecto y permite importar funciones de Y? y clases de Zetariano. Las palabras principales incluyen MAIOR>, FINIS;, VARIABILES>, esto, series, novus, null y operaciones de entrada/salida << y >>.

Conjunto	Elementos compartidos	Diferencias relevantes
Operadores	+, -, *, /, ==, !=, <, >, <=, >=	Zetariano también incorpora %, ternario y asignación compuesta
Lógicos	&&, || y negación	Y? y PigLatin usan !; Zetariano utiliza !
Agrupación	(), [], {}, punto, coma, punto y coma	Y? usa indentación adicional
Comentarios	// y /*...*/ en Y? y Zetariano	PigLatin admite además ##...##

AST y tabla de símbolos

AnalizadorArchivo entrega contextos de ANTLR a los builders específicos YAstBuilder, ZAstBuilder y PAstBuilder. Estos crean nodos propios del proyecto bajo ast, ast.zetariano y ast.piglatin. La ubicación de origen se conserva por medio de PosicionFuente.

Componente	Representación	Uso
ProgramaAst	Lista de elementos y ubicación	Raíz común del AST de archivo
FuncionAst / EstructuraAst	Funciones y tipos definidos por usuario en Y?	Tipado y generación de cuartetas
ZAst	Clase, método, atributo, constructor, expresión	Herencia y objetos Zetariano
PAst	Programa, declaración, sentencia, expresión	Código principal e imports PigLatin
TablaSimbolos / Ambito / Simbolo	Identificadores y relaciones de alcance	Búsqueda, duplicados y resolución de tipos

Los AST no son el árbol sintáctico que devuelve directamente ANTLR: representan información útil para las siguientes etapas y no incluyen cada token de puntuación. La tabla de símbolos permite interpretar referencias en su ámbito y reconocer variables, tipos y funciones definidos previamente.

Análisis semántico y compatibilidad de tipos

Existen tres analizadores independientes. AnalizadorSemanticoY revisa funciones y estructuras; AnalizadorSemanticoZ valida clases, firmas, tipos y herencia; AnalizadorSemanticoPig verifica referencias importadas, variables, objetos, llamadas y el archivo principal. Los errores se guardan como Diagnostico con tipo, severidad, archivo y posición.

Equivalencia básica de tipos

Tipo abstracto	Y?	Zetariano	PigLatin
Entero	entero	int	numerus
Decimal	flotante	double	decimalis
Carácter	caracter	char	littera
Booleano	bool	boolean	bool / verum / falsus
Cadena	cadena	String	textum
Referencia	estructura / arreglo	clase / arreglo	objeto / arreglo

Resultado de expresiones habituales

Operación	Y?	Zetariano	PigLatin
entero + entero	entero	int	numerus
entero + decimal	flotante	double	decimalis
decimal × entero	flotante	double	decimalis
cadena + cadena	Se requiere comprobar el operador	String	textum
cadena + entero	No se documenta como promoción numérica	Concatenación	Concatenación
comparación numérica	bool	boolean	bool
&& / ||	Solo booleanos	Solo booleanos	Solo booleanos

Esta tabla sintetiza las reglas de tipo relevantes observadas en las implementaciones. No debe leerse como garantía de que cualquier combinación de operadores está admitida: cada analizador valida operador, tipos, referencias y contexto.

Compatibilidad en asignaciones

Caso	Y?	Zetariano	PigLatin
entero → decimal	Permitido por ampliación numérica	Permitido: int → double	Permitido: numerus → decimalis
decimal → entero	No permitido por reducción	No permitido por reducción	No permitido por reducción
carácter → entero	Permitido en conversiones numéricas	Depende de promoción numérica	Permitido: littera → numerus
padre ← hija	No aplica a clases Z	Permitido en clases relacionadas	Permitido en objetos relacionados
hija ← padre	No aplica a clases Z	Rechazado sin conversión válida	Rechazado
objeto ← null	Según tipo de referencia	Admitido en referencias	Admitido en objetos
primitivo ← null	No permitido	No permitido	No permitido

La promoción numérica se realiza atendiendo al tipo inferido de toda la expresión, no únicamente a si un operando fue escrito como literal. Ejemplo: un parámetro double puede compararse con el entero 0.

Herencia, polimorfismo y objetos

En Zetariano, ValidadorHerenciaZ verifica la existencia de padres, ausencia de ciclos, firmas de métodos sobrescritos, @Override, compatibilidad del retorno y visibilidad. JerarquiaClasesZ centraliza la relación padre-hija y su resolución. La visibilidad sin modificador se trata como privada en esta implementación, según la aclaración utilizada para el banco de pruebas.

Modificador	Regla aplicada en el proyecto
public	Accesible en el contexto permitido por el enlace entre clases
protected	Accesible desde subclases y la propia clase
private	Restringido a la clase que declaró el miembro
default	Tratado como private en esta práctica
@Override	Obligatorio para identificar la redefinición de un método heredado

```
public class Hija extends Padre {
    @Override
    public int valor() { return 222; }
}
```

La planificación de memoria considera los atributos heredados al reservar el objeto. El despacho dinámico conserva el tipo real de la instancia para seleccionar métodos sobrescritos cuando una referencia de tipo padre apunta a una hija. En cambio, super.metodo() mantiene la resolución explícita del miembro padre.

Caso real: el banco de la auxiliar exigió cambiar movimientos de acceso default a protected en Cuenta.z, conservando la regla de privacidad. Posteriormente se corrigió la generación de llamadas sobre objetos ubicados en arreglos, por ejemplo cuentas[i].getNumero().

Cuartetas, Stack y Heap

El programa intermedio se representa en ProgramaIntermedio como secuencia de Cuarteta y OperadorCuarteta. GeneradorCuartetasY, GeneradorCuartetasZ y GeneradorCuartetasPig emiten operaciones a partir de cada AST y GeneradorIntermedioProyecto coordina los tres lenguajes.

Concepto	Clases asociadas	Aplicación
Cuarteta	Cuarteta, OperadorCuarteta	Operador, argumentos, resultado y control de flujo
Stack	MarcoStackProyecto, SlotStackProyecto	Parámetros, locales, temporales y retornos
Heap	LayoutHeapProyecto, PlanificadorMemoriaProyecto	Objetos, arreglos, cadenas y estructuras
Bajadores	BajadorMemoriaProyecto, BajadorLlamadasProyecto	Lecturas, escrituras, direcciones y llamadas
Especializados	BajadorArreglosProyecto, BajadorCadenasProyecto	Arreglos, cadenas e inicialización

La asignación de memoria ocurre después de construir cuartetas. La clase hija conserva sus atributos y los de sus antecesores. Las celdas del Heap son direccionadas mediante offsets; el Stack almacena información de las funciones y sus activaciones.

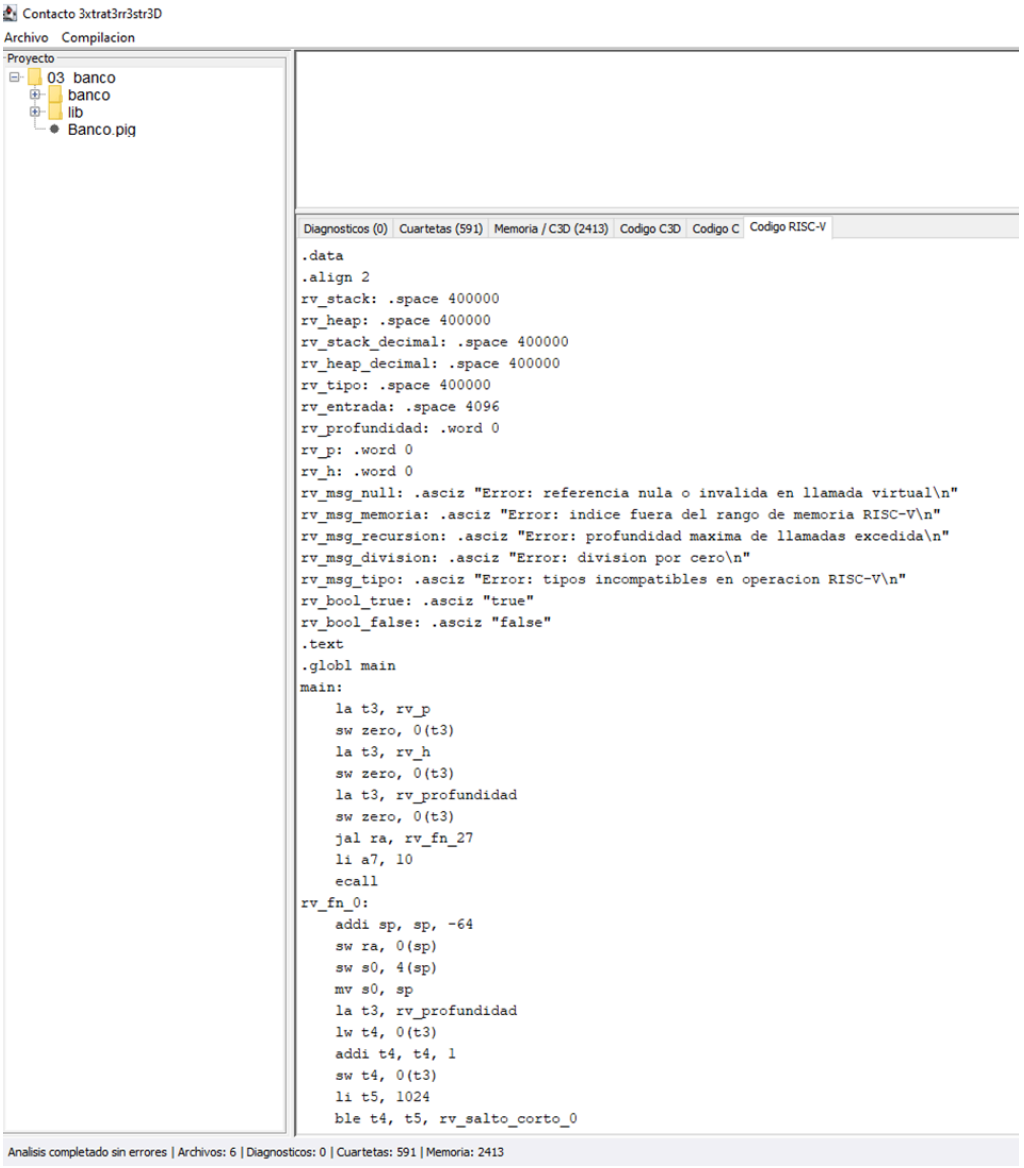


Figura 3. Banco de la auxiliar analizado sin diagnósticos, con 591 cuartetas y 2413 operaciones de memoria/C3D.

Generación de código C y RISC-V

Backend C y C3D

GeneradorCodigoTresDirecciones construye una representación textual del código intermedio. GeneradorCodigoC transforma la representación de memoria a un programa C que puede verificarse con GCC. La salida C fue utilizada como comprobación adicional durante el desarrollo, pero no sustituye el objetivo de ensamblador funcional.

Backend RISC-V

GeneradorRiscV recibe ProgramaIntermedio después de los bajadores de memoria y genera un archivo .s. Usa la sección .data para Stack, Heap, tablas auxiliares de tipos y mensajes de error, y .text para funciones, saltos, llamadas y rutinas auxiliares. La implementación trabaja con RISC-V de 32 bits y extensión de punto flotante simple (RV32IMF); la ejecución se validó con RARS.

Elemento	Uso técnico
rv_stack / rv_heap	Regiones de memoria simuladas
rv_stack_decimal / rv_heap_decimal	Almacenamiento asociado al tratamiento de decimales
rv_tipo	Información auxiliar para tipo real de objetos
rv_p / rv_h	Posiciones lógicas de Stack y Heap
jal, ret, saltos	Invocación de subprogramas y control de flujo
ecall	Entrada, impresión y finalización en RARS
Rutinas de validación	Mensajes de error de referencia, memoria, división y recursión

En programas extensos se evitan saltos condicionales fuera de rango usando una condición invertida, una etiqueta cercana y un salto incondicional al destino final. Esta corrección se validó en el caso de herencia de tres niveles.

```
java -jar "$env:USERPROFILE\Downloads\rars1_6.jar" nc .\programa.s
```

La salida decimal puede mostrar ligeras diferencias de precisión frente a C porque el backend emplea punto flotante simple. En pruebas de concatenación decimal se documentó el formato visible con tres decimales.

Interfaz gráfica y gestión de proyectos

VentanaPrincipal está implementada con Swing. Presenta un árbol lateral de archivos, un área central de edición y un panel inferior de resultados. La clase ColoreadorAntlr realiza coloreado de código y GestorProyecto maneja lectura y escritura. El editor admite archivos de los tres lenguajes de entrada.

Pestaña	Información mostrada
Diagnósticos	Severidad, tipo, archivo, línea, columna y mensaje
Cuartetas	Representación intermedia del proyecto
Memoria / C3D	Operaciones de Stack, Heap y memoria
Código C3D	Salida textual de tres direcciones
Código C	Programa C generado
Código RISC-V	Ensamblador generado, disponible para exportación

El menú Archivo administra proyectos y documentos; el menú Compilación permite analizar y guardar el código generado. La interfaz muestra explícitamente cuando el backend RISC-V no puede representar una operación, evitando presentar silenciosamente una salida incompleta.

Diagnósticos, pruebas y evidencia

Tipo de diagnóstico	Detección	Tratamiento
Léxico	ANTLR4 lexer + RecolectorErroresAntlr	Se identifica token o secuencia inválida
Sintáctico	ANTLR4 parser	Se informa regla inesperada o estructura incompleta
Semántico	AnalizadorSemanticoY/Z/Pig y validadores	Referencias, tipos, firmas, accesibilidad y contexto
Proyecto	CompiladorProyecto	Falta de archivos o cantidad incorrecta de .pig
Generación	Backends	Operación no implementada o restricción técnica

Comprobaciones registradas durante el desarrollo: 255 pruebas automáticas sin fallos en la versión de saltos largos; se añadieron posteriormente dos casos de regresión sobre llamadas a métodos en arreglos, cuya ejecución Maven completa no consta en las evidencias facilitadas. El banco de la auxiliar se analizó con 0 diagnósticos, 591 cuartetas y 2413 operaciones de memoria, y su programa RISC-V se ejecutó en RARS con saldos finales coherentes.

El banco de la auxiliar ejercitó clases de cuenta, herencia, saldos decimales, retiros, sobregiros, búsquedas y bitácora. La siguiente evidencia es la ejecución del archivo programa.s guardado en la carpeta del banco.

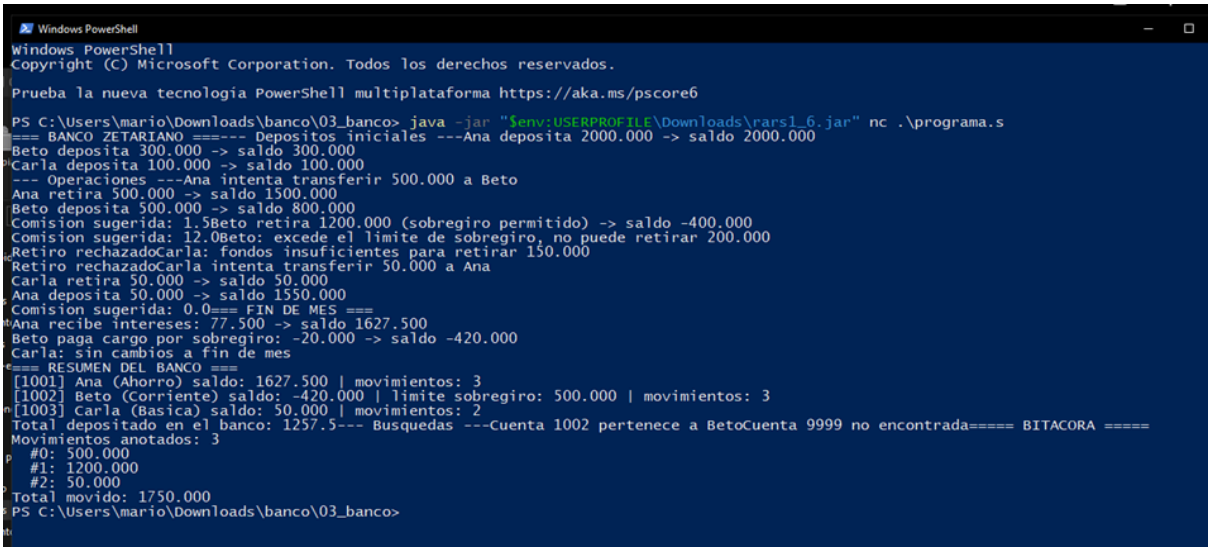


Figura 4. Ejecución RISC-V del banco en RARS; resumen de cuentas y operaciones mostrado en PowerShell.

Regresión automática

```
mvn clean test
 powershell -ExecutionPolicy Bypass -File .\validar-riscv-etapas-5-6.ps1 -OmitirMaven
```

El script permite exportar doce proyectos integrales, ejecutar los ensambladores en RARS y comparar las salidas con valores esperados. En la validación compartida reportó 12 correctas y 0 incorrectas. No equivale a demostrar que todos los programas posibles estén cubiertos.

Instalación, ejecución y mantenimiento

9. Instalar JDK 21 y comprobar con java -version; instalar Maven y comprobar con mvn -version.
10. Abrir en IntelliJ IDEA el directorio contacto-extraterrestre, donde está el archivo pom.xml.
11. Ejecutar mvn clean test para regenerar ANTLR4, compilar y ejecutar las pruebas.
12. Iniciar con mvn exec:java o mediante la clase com.compi2.contacto.Aplicacion.
13. Abrir una carpeta con exactamente un archivo .pig y los .y/.z requeridos; seleccionar Compilación > Analizar proyecto.
14. Guardar el código RISC-V en programa.s; ejecutarlo con RARS para comprobar la salida.
15. Repetir los tests luego de modificar gramáticas, semántica, bajadores o el generador.

Puntos de mantenimiento

Modificar primero las gramáticas .g4 para cambiar sintaxis; regenerar las clases ANTLR4 mediante Maven.
Actualizar el builder correspondiente si cambia la forma de un nodo del AST.
Ajustar los análisis semánticos y las pruebas de compatibilidad de tipos antes de producir cuartetas.
Si se introduce un operador nuevo, implementar la cadena completa: IR, memoria y backends.
Conservar pruebas de regresión del banco y de los casos RISC-V para evitar repetir errores de llamadas y saltos.

Especificación de palabras reservadas, símbolos y gramática de los tres lenguajes

Esta tabla presenta de manera comparativa los elementos principales de Y?, Zetariano y PigLatin, utilizando el mismo formato de la tabla de compatibilidad de tipos.

Categoría	Y?	Zetariano	PigLatin
Secciones principales	%estructuras, %funciones	Declaraciones de clases	VARIABILES>, MAIOR>, FINIS;
Tipos enteros	entero	int	numerus
Tipos decimales	flotante	double	decimalis
Cadenas de texto	cadena	String	textum
Caracteres	caracter	char	littera

Booleanos	bool	boolean	bool
Valores verdaderos	verdadero	true	verum
Valores falsos	falso	false	falsus
Declaración de variables	Identificador y tipo declarado	Tipo seguido del identificador	esto
Declaración de arreglos	Dimensiones con []	Dimensiones con []	series
Estructuras	estructura	class	Tipos de objeto importados
Herencia	No aplica	extends	Compatibilidad entre objetos padre e hija
Encapsulamiento	No aplica	public, private, protected	Depende de la clase importada
Polimorfismo	No aplica	@Override	Llamadas a métodos sobrescritos
Creación de objetos	No aplica	new	novus
Referencias nulas	Según el tipo admitido	null	null
Referencia al objeto actual	No aplica	this	Acceso mediante el objeto declarado
Referencia a la clase padre	No aplica	super	Mediante los métodos heredados
Declaración de funciones	definir	Métodos dentro de clases	Funciones importadas de otros lenguajes
Retorno de funciones	retornar	return	reddere en funciones compatibles
Condicionales	si, entonces, sino, contrario	if, else	si, aliter, finis

Selección múltiple	elegir, caso, siempre	switch, case, default	Condicionales con si y aliter
Ciclo mientras	mientras	while	dum
Ciclo hacer-mientras	hacer	do, while	facere, dum
Ciclo para	para	for	per
Interrupción de ciclos	romper	break	interrumpe
Continuación de ciclos	continuar	continue	perge
Entrada de datos	leer	readln	<<
Salida de datos	imprimir	print, println	>>
Operadores aritméticos	+, -, *, /	+, -, *, /, %	+, -, *, /
Operadores relacionales	==, !=, <, >, <=, >=	==, !=, <, >, <=, >=	==, !=, <, >, <=, >=
Operadores lógicos	&&, ||, !	&&, ||, !	&&, ||, non
Incremento y decremento	++, --	++, --	++, --
Operadores de asignación	=	=, +=, -=, *=, /=	=
Agrupación	(), [], {}	(), [], {}	(), [], {}
Separadores	:, ;, ,, .	:, ;, ,, .	:, ;, ,, .
Delimitación de bloques	Indentación	Llaves {}	Bloques y palabras de cierre
Gramática de estructuras	estructura Nombre: con atributos indentados	class Nombre { ... }	Tipos definidos o importados

Gramática de herencia	No aplica	class Hija extends Padre { ... }	Declaración de referencias a clases compatibles
Gramática de objetos	No aplica	Tipo objeto = new Clase();	esto objeto : novus Clase() : Tipo;
Gramática de variables	Identificador, tipo y valor según declaración	int numero = 10;	esto numero : numerus 10;
Gramática de funciones	definir nombre(...): seguido de instrucciones indentadas	public int metodo() { ... }	Llamada a función importada
Gramática de entrada/salida	imprimir(valor)	println(valor);	>> valor;
Análisis sintáctico	ANTLR4: YLexer.g4 y YParser.g4	ANTLR4: ZetarianoLexer.g4 y ZetarianoParser.g4	ANTLR4: PigLatinLexer.g4 y PigLatinParser.g4

Compatibilidad de tipos de datos para Y?, Zetariano y PigLatin

Esta tabla muestra el tipo resultante de distintas operaciones y las conversiones permitidas por el análisis semántico del compilador.

Operación o conversión	Y?	Zetariano	PigLatin
Entero + Entero	entero	int	numerus
Entero + Decimal	flotante	double	decimalis
Decimal + Decimal	flotante	double	decimalis
Entero - Decimal	flotante	double	decimalis
Entero * Decimal	flotante	double	decimalis

Entero / Entero	entero	int	numerus
Entero / Decimal	flotante	double	decimalis
Carácter + Carácter	caracter	int	numerus
Carácter + Entero	entero	int	numerus
Carácter + Decimal	flotante	double	decimalis
Cadena + Entero	cadena	String	textum
Cadena + Decimal	cadena	String	textum
Cadena + Cadena	cadena	String	textum
Cadena + Booleano	cadena	String	textum
Cadena - Entero	Error semántico	Error semántico	Error semántico
Cadena * Entero	Error semántico	Error semántico	Error semántico
Entero + Booleano	Error semántico	Error semántico	Error semántico
Entero < Decimal	bool	boolean	bool
Entero == Decimal	bool	boolean	bool
Cadena == Cadena	bool	boolean	bool
Booleano && Booleano	bool	boolean	bool
Booleano || Booleano	bool	boolean	bool

Booleano && Entero	Error semántico	Error semántico	Error semántico
Negación de Booleano	bool	boolean	bool
Asignar Entero a Decimal	Permitido	Permitido	Permitido
Asignar Decimal a Entero	Error semántico	Error semántico	Error semántico
Asignar Carácter a Entero	Permitido	Permitido	Permitido
Asignar Entero a Carácter	Error semántico	Error semántico	Error semántico
Asignar Booleano a Entero	Error semántico	Error semántico	Error semántico
Asignar null a objeto	No aplica	Permitido	Permitido
Asignar objeto hijo a variable padre	No aplica	Permitido	Permitido
Asignar objeto padre a variable hija	No aplica	Error semántico	Error semántico
Asignar estructura a estructura del mismo tipo	Permitido	No aplica	Según el tipo declarado

Trazabilidad con el enunciado

Requisito Proyecto 2	Dónde se implementa o verifica
Análisis léxico, sintáctico y semántico	Gramáticas ANTLR4; AnalizadorArchivo; analizadores semánticos
Interfaz amigable	VentanaPrincipal, PanelEditor, pestañas de resultados
Coloreado en tiempo real sin librería de coloreado	editor/ColoreadorAntlr.java
Árbol de trabajo; abrir, guardar y descargar	GestorProyecto y funciones del árbol en VentanaPrincipal
Encapsulamiento/herencia/polimorfismo	JerarquiaClasesZ, ValidadorHerenciaZ, despacho dinámico
Objetos PigLatin	PAst, AnalizadorSemanticoPig y llamadas a constructores
Representación intermedia y memoria	ir/* y memoria/*
RISC-V funcional	GeneradorRiscV; RARS; banco y doce pruebas integrales
Manual de usuario y técnico	Documentación complementaria al repositorio
Tecnologías, clases, gramáticas y compatibilidad	Secciones 2, 4–8 de este documento

Conclusiones

- La separación entre ANTLR4, AST, análisis semántico, IR, memoria y backends permite corregir un problema de compilación sin sustituir todo el proyecto.
- La ejecución del banco en RARS verifica en un escenario integrado el manejo de decimales, objetos, herencia, arreglos y llamadas.
- La documentación debe mantenerse sincronizada con el repositorio; cualquier cambio de gramática o representación de memoria debe acompañarse de pruebas y actualización del manual.